Week 3

Physics-Informed Neural Networks

Velioglu et al. (2025)

Model Approaches

Process models are classified by how much they rely on physical knowledge vs data:

Type	Also called	Description
White-box	First-principles	Derived entirely from physical/chemical laws
Black-box	Data-driven	Learned from data, no physical knowledge
Gray-box	Hybrid	Combines both

Hybrid Model Architectures

Serial

The data-driven model *corrects* the output of the mechanistic model:

$$\hat{y} = \underbrace{f_{\text{mech}}(\mathbf{x})}_{\text{base}} + \underbrace{\hat{d}}_{\text{correction}}$$

- Mechanistic model runs first; NN adds a residual
- OBL is a serial model — Kalman filter corrects LARS online
- Correction is interpretable (same units as y)

Parallel

Both models contribute directly to the prediction:

$$\hat{y} = \alpha\, f_{\text{mech}}(\mathbf{x}) + (1-\alpha)\, f_{\text{NN}}(\mathbf{x})$$

- Mechanistic and NN models run side by side
- Blending weight α can be fixed or learned
- Requires a full mechanistic model at inference

PINN

NN is the sole prediction model; physics enter as loss terms during training:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_1 \mathcal{L}_{\text{physics}} + \lambda_2 \mathcal{L}_{\text{init}}$$

- No mechanistic model needed at inference
- Physics constrain the NN without requiring complete knowledge
- Can infer unmeasured states

Physics-Informed Neural Networks

Key idea (Raissi et al., 2019): train a neural network not just to fit data, but to *obey physical laws* — by adding a physics residual term to the loss.

- The NN acts as the **sole prediction model**
- Physical laws appear as **soft constraints** — residuals in the loss function
- No separate mechanistic sub-model at inference time

Velioglu et al. (2025) extend this to dynamic process models described by semi-explicit differential–algebraic equations (DAEs), with:

- Incomplete physical knowledge (some constitutive equations unknown)
- Scarce process data (some states unmeasured)
- Varying initial conditions and control inputs as NN inputs

1. M. Raissi, P. Perdikaris and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations", Journal of Computational Physics, 378, 686–707 (2019). doi:10.1016/j.jcp.2018.10.045

2. M. Velioglu, S. Zhai, S. Rupprecht, A. Mitsos, A. Jupke and M. Dahmen, "Physics-informed neural networks for dynamic process operations with limited physical knowledge and data", Computers & Chemical Engineering, 192, 108899 (2025). doi:10.1016/j.compchemeng.2024.108899

Math Preliminaries — DAE System

A semi-explicit differential–algebraic equation (DAE) system (Eqs. 1a–1b):

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t),\, \mathbf{y}(t),\, \mathbf{u}) \tag{1a}$$

$$\mathbf{0} = \mathbf{g}(\mathbf{x}(t),\, \mathbf{y}(t),\, \mathbf{u}) \tag{1b}$$

- $\mathbf{x}$: **differential states** (have time derivatives; e.g. concentrations, temperatures)
- $\mathbf{y}$: **algebraic states** (no time derivatives; e.g. reaction rates, equilibrium quantities)
- $\mathbf{u}$ — **control inputs** (manipulated variables)

States are further classified as:

Superscript	Meaning
$\mathbf{x}^m,\ \mathbf{y}^m$	**Measured** — data available for training
$\mathbf{x}^u,\ \mathbf{y}^u$	**Unmeasured** — no direct data; must be inferred

Goal: estimate $\mathbf{x}^u$ and $\mathbf{y}^u$ from data on $\mathbf{x}^m$ and $\mathbf{y}^m$, using partial knowledge of $\mathbf{f}$ and $\mathbf{g}$.

PINN Architecture

The neural network $\mathrm{NN}_{\mathbf{w},\mathbf{b}}$ maps:

$$\left(t,\ \mathbf{x}^m(t_0),\ \mathbf{u}\right) \longrightarrow \left(\hat{\mathbf{x}}(t),\ \hat{\mathbf{y}}(t)\right)$$

— predicting **all** states (measured and unmeasured) as functions of time.

Three sources of supervision:

Loss term	What it enforces
$\mathrm{MSE}_{\mathrm{data}}$	NN predictions of measured states match data
$\mathrm{MSE}_{\mathrm{physics}}$	NN predictions satisfy the known ODE/DAE residuals
$\mathrm{MSE}_{\mathrm{init}}$	NN predictions at t_0 match known initial conditions

The time derivative $\dot{\hat{\mathbf{x}}}(t)$ needed for $\mathrm{MSE}_{\mathrm{physics}}$ is computed via **automatic differentiation** of the NN output with respect to t — in TensorFlow, using `tf.GradientTape`.

$$MSE_{total} = MSE_{data} + \lambda_1 MSE_{physics} + \lambda_2 MSE_{init}, \tag{2a}$$

$$MSE_{data} = \frac{1}{n_{x^m} N_d} \sum_{j=1}^{N_d} (\hat{\mathbf{x}}^m(t_j) - \mathbf{x}^m(t_j))^2 + \frac{1}{n_{y^m} N_d} \sum_{j=1}^{N_d} (\hat{\mathbf{y}}^m(t_j) - \mathbf{y}^m(t_j))^2, \tag{2b}$$

$$MSE_{physics} = \frac{1}{n_x N_e} \sum_{j=1}^{N_e} \left(\dot{\hat{\mathbf{x}}}(t_j) - \boldsymbol{f}(\hat{\mathbf{x}}(t_j), \hat{\mathbf{y}}(t_j), \mathbf{u}_j)\right)^2 + \frac{\lambda_g}{n_y N_e} \sum_{j=1}^{N_e} \left(\boldsymbol{g}(\hat{\mathbf{x}}(t_j), \hat{\mathbf{y}}(t_j), \mathbf{u}_j)\right)^2, \tag{2c}$$

$$MSE_{init} = \frac{1}{n_{x^m} N_i} \sum_{j=1}^{N_i} (\hat{\mathbf{x}}_j^m(t_0) - \mathbf{x}_j^m(t_0))^2 \tag{2d}$$

Automatic Differentiation

The physics loss requires $\dot{\hat{\mathbf{x}}}(t) = \frac{d\hat{\mathbf{x}}}{dt}$ — the time derivative of the NN output.

AD computes this exactly by applying the chain rule through the network graph:

$$\frac{d\hat{\mathbf{x}}}{dt} = \frac{\partial \hat{\mathbf{x}}}{\partial \mathbf{h}_L} \cdot \frac{\partial \mathbf{h}_L}{\partial \mathbf{h}_{L-1}} \cdots \frac{\partial \mathbf{h}_1}{\partial t}$$

where $\mathbf{h}_\ell$ are the hidden layer activations.

- In **TensorFlow**, computed inside a `GradientTape` context:

```
with tf.GradientTape() as tape:
    tape.watch(t)
    x_pred = model(t)           # NN forward pass
x_dot = tape.gradient(x_pred, t)   # dx/dt via AD
```

A distinguishing feature of PINNs: the ODE residual involves $\dot{\hat{\mathbf{x}}}$, which requires differentiating through the network.

Van de Vusse CSTR

A classic benchmark for nonlinear process control (van de Vusse, 1964):

$$A \xrightarrow{k_1} B \xrightarrow{k_2} C \qquad 2A \xrightarrow{k_3} D$$

B is the desired product; C and D are unwanted byproducts.

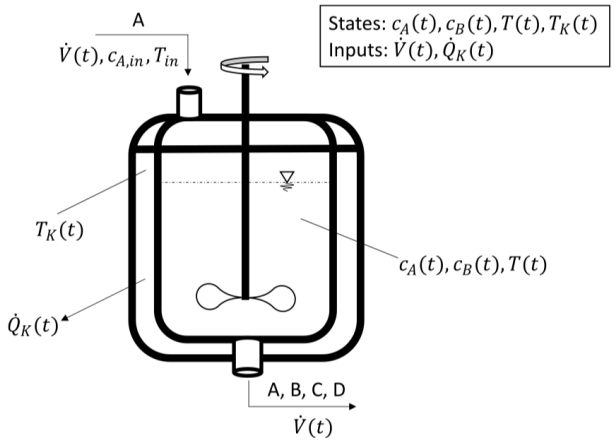

Fig. 3. Schematic representation of the van de Vusse CSTR.

Differential states $\mathbf{x}$:

- c_A, c_B — concentrations of A and B (mol/L)
- T — reactor temperature (K)
- T_K — cooling jacket temperature (K)

Manipulated inputs $\mathbf{u}$:

- $D = \dot{V}/V_R$ — dilution rate (1/h)
- $\dot{Q}_K$ — coolant heat removal rate (kJ/h)

Rate constants follow Arrhenius: $k_i(T) = k_{i0} \exp(E_{a,i}/T)$ — but in **PINN-C**, k_1, k_2, k_3 are treated as **unknown** functions to be inferred.

Table 3
Physical knowledge and output configuration (measured and unmeasured process states) for the Van de Vusse CSTR PINN models. In PINN-C, T and T_K can also be unmeasured depending on the case study (cf. Section 3.4). The time dependence of the states is not shown explicitly for brevity. The manipulated variables $\frac{\dot{V}}{V_R}$ and $\dot{Q}_K$ are the step-wise constant controls which we, for the sake of a simple implementation, keep constant throughout the investigated process duration.

Model name	Physics knowledge	Physics equations	Measured process states	Unmeasured process states
Vanilla ANN	None	None	c_A, c_B, T, T_K	None
PINN-A	Mole balances with net reaction rates	$\dot{c}_A = \frac{\dot{V}}{V_R}(c_{A,in} - c_A) + r_A$ $\dot{c}_B = -\frac{\dot{V}}{V_R}c_B + r_B$	c_A, c_B, T, T_K	r_A, r_B
PINN-B	Mole and energy balances with individual reaction rates	$\dot{c}_A = \frac{\dot{V}}{V_R}(c_{A,in} - c_A) - r_1 - r_3$ $\dot{c}_B = -\frac{\dot{V}}{V_R}c_B + r_1 - r_2$ $\dot{T} = \frac{\dot{V}}{V_R}(T_{in} - T) + \frac{k_w A_R}{\rho C_p V_R}(T_K - T) - \frac{1}{\rho C_p}(r_1 \Delta H_{AB} + r_2 \Delta H_{BC} + r_3 \Delta H_{AD})$ $\dot{T}_K = \frac{1}{m_K C_{pK}}(\dot{Q}_K + k_w A_R (T - T_K))$	c_A, c_B, T, T_K	r_1, r_2, r_3
PINN-C	Mole and energy balances with reaction rate expressions (without Arrhenius' law)	$\dot{c}_A = \frac{\dot{V}}{V_R}(c_{A,in} - c_A) - k_1 c_A - k_3 c_A^2$ $\dot{c}_B = -\frac{\dot{V}}{V_R}c_B + k_1 c_A - k_2 c_B$ $\dot{T} = \frac{\dot{V}}{V_R}(T_{in} - T) + \frac{k_w A_R}{\rho C_p V_R}(T_K - T) - \frac{1}{\rho C_p}(k_1 c_A \Delta H_{AB} + k_2 c_B \Delta H_{BC} + k_3 c_A^2 \Delta H_{AD})$ $\dot{T}_K = \frac{1}{m_K C_{pK}}(\dot{Q}_K + k_w A_R (T - T_K))$	c_A, c_B, T, T_K	k_1, k_2, k_3

How PINN-C Estimates Unknown States

The NN outputs **all seven states** simultaneously: $\hat{c}_A,\ \hat{c}_B,\ \hat{T},\ \hat{T}_K,\ \hat{k}_1,\ \hat{k}_2,\ \hat{k}_3$.

- **MSE_data** anchors $\hat{c}_A, \hat{c}_B, \hat{T}, \hat{T}_K$ to measurements
- **MSE_physics** penalises violation of Eqs. 3a–3d at measurement points

At each point, **automatic differentiation** gives $\dot{\hat{c}}_A, \dot{\hat{c}}_B, \dot{\hat{T}}, \dot{\hat{T}}_K$ exactly. With D and $\dot{Q}_K$ known, the ODE residuals contain $\hat{k}_1, \hat{k}_2, \hat{k}_3$ as the **only free quantities**. For example, Eq. 3a:

$$\underbrace{\dot{\hat{c}}_A}_{\text{AD}} = D(c_{A,in} - \hat{c}_A) - \underbrace{\hat{k}_1}_{\text{unknown}}\ \hat{c}_A - \underbrace{\hat{k}_3}_{\text{unknown}}\ \hat{c}_A^2$$

The optimizer finds $\hat{k}_i(t)$ that balances all four equations simultaneously — an **inverse problem embedded in the loss**. **No ODE integration is required.**

Why Not Just Fit the White-Box Model?

With the full mechanistic model (Eqs. 3a–3d + Arrhenius), one could estimate k_{i0} and $E_{a,i}$ by minimising the residuals between simulated and measured trajectories. This is classical **parameter estimation**. Several reasons to prefer the PINN approach:

Issue	White-box parameter estimation	PINN-C
Functional form of k_i	Must assume Arrhenius $k_{i0}e^{E_{a,i}/T}$	$k_i(t)$ inferred as a free function — no assumed form
Unknown constitutive equations	Cannot proceed (PINN-A, B have no rate law at all)	Not required
Computational cost	Requires solving the ODE at every optimizer step	Trains on data points — no ODE integration

White-box fitting assumes you know the model structure completely. The PINN is designed for the case where you know the *balance equations* but not the *constitutive equations* for the unmeasured quantities.